Students' Names:	Shahram Shafiq Haris Shafiq
Roll Numbers:	24i-2541 24i-0746
Course Name:	Object Oriented Programming
Program:	BSCS
Semester/Section:	Fall-25 /A
Lab Instructor:	Ms. Mahnoor Javed
Project Title:	Galaxy Wars — the Object-Oriented Space Battle
Date of Submission:	13/09/2025
Institute:	FAST NUCES

Galaxy Wars: 2D Space Shooter Game

Project Report

1. Overview and Motivation

Project Overview

Galaxy Wars is a space shooter game in 2D created using C++ with a framework of SFML (Simple and Fast Multimedia Library). The player of the game is tasked with flying a spaceship across three increasingly challenging levels, destroying various types of enemies, gathering power-ups and scoring high points. In this project, we have thoroughly applied and implemented the principles of OOP such as inheritance, polymorphism, encapsulation, abstraction, and dynamic memory management [1].

Core Gameplay:

The gamer is in charge of a spaceship by using arrow keys with two shooting modes (manual shooting through Space Bar and automatic fire through the F key). There are three types of enemies, which are the: Basic Enemies descending vertically, Fast Enemies, who fall twice as fast, and Pattern Enemies, moving in zigzag sine waves, and shoot back. The formations of the enemies are horizontal, V-formations, and diagonal waves forming strategic challenges for the players [1].

The difficulty progression system is characterized by three levels that are activated by score levels (Level 2 at 500 points, Level 3 at 1500 points). Level 3 also has a 120-second survival clock to provide a sense of urgency. The game features three types of power-ups that are gameplay benefits: Shield (temporary invulnerability with visual bubble), Fire Rate (cooldown reduced to 0.08s to fire continuously), and Repair (30 health restoration with particle effects) [1].

Technical Architecture:

The project has 20 classes with the distribution implemented in 9 header files (GameException.h, Resource.h, BoundingBox.h, GameObject.h, GameObjectArray.h, Ships.h, GameObjects.h, Managers.h, GameEngine.h) and main.cpp. The system involves template-based dynamic arrays used to store game objects, abstract base classes to provide polymorphic behavior, proper copy constructors to provide deep copying. The game engine runs at 60 frames per second, handles 10 different game states (splash screen, menus, gameplay, level transitions, game over) and uses axis-aligned bounding to perform collision detection [1].

The format of files read by file input and output is either text (scores.txt, with values separated by spaces) or binary (scores.dat, length-prefixed strings). Exception handling encloses file

operations that could be throwing custom GameException objects which are safely caught in main()[1].

Main Motivation

The first reason was the desire to prove that we understand OOP principles by creating a fully functional, complete-fledged game application. This project was contrary to single concept exercises throughout our course and semester, where one would apply single concepts individually, as now we were expected to apply several concepts of OOP at the same time and solve real-life problems in the sector of programming [1].

Educational Goals:

We wanted to have some real-life experience in designing complex class hierarchies with multilevel inheritance with abstract base classes and specialized derived classes. The project allowed the realization of the concept of polymorphism and dynamic binding when dealing with game object handling where the base class pointer invokes overridden methods at runtime. Implementing the GameObjectArray template class taught us the principles of generic programming that ensures type safety by instantiation at compile time [1].

Exception handling and file I/O functionality were useful experiences in data persistence and careful programming. Using SFML as a graphics and audio subsystem showed the concept of external libraries being used with object-oriented designs. The concepts of software architecture were naturally learned as a result of the separation of concerns in more than one file and in more than one class which was organized in terms of functionality[1].

Technical Significance:

The space shooter genre naturally maps to OOP design patterns with spaceships, enemies, projectiles, and power-ups being different entities that are similar in their properties but different in their behavior. They act like different objects in terms of programming. This gave a perfect model of presenting the hierarchies of inheritance, polymorphic behavior and composition relationships [1].

The fact that it featured progressive difficulty levels, wave-based spawning with formations, the use of visual particle effects, and dynamic music, challenged us beyond basic requirements of a high-quality game or applying OOP concepts. The project helped to develop skills of debugging complex multi-file systems as we have multiple different header files, and performance optimization based on efficient collision detection[1].

2. Class Diagram

Architecture Overview

The Galaxy Wars game framework has 20 classes organized in inheritance hierarchies showing single inheritance, multilevel inheritance, and composition relationships. The design focuses on

separation of concerns, appropriate abstraction levels, and professional object-oriented practices [1].

Class Hierarchy Summary

Foundation Classes:

The GameException class provides custom exception handling for game-specific errors with an errormsg string accessible through the getMessage() method this makes sure that the variable remain out of access to the player (encapsulation). The Resource class encapsulates scoring with value accessors and operator overloading for combining resources. The BoundingBox struct represents collision rectangles with public members x, y, width, height and an intersects() method for collision testing[1].

Core Abstract Base:

GameObject serves as the abstract base class that defines the common interface for all game objects with protected members (sprite, texture, x, y, speed, active flag, width, height, scaleX, scaleY). It declares pure virtual update() which forces derived classes to implement movement logic, virtual draw() method with default rendering, getBounds() for collision boxes, and updateCollisionFromTexture() for dynamic dimension extraction[1].

Inheritance Hierarchies:

Ship is an intermediate class that extends GameObject adding health management with a protected int health member and methods takeDamage(), getHealth(), setHealth(). Player extends Ship with complex state management including bullets pointer, collectedresources, shield tracking, fire control variables (firecooldown, firetimer, fireRateDuration), velocity tracking (velX, velY), and particle system for visual effects[1].

Enemy extends Ship with downward movement and static enemycount which is a counter tracking total enemies. FastEnemy extends Enemy with increased speed (4.0 vs 2.0). PatternEnemy extends Enemy adding sine wave movement with horizontal displacement and shooting capability. BossEnemy extends Enemy with higher health (100), slower speed (1.5), and larger scale (0.6)[1].

Projectile Classes:

Projectile represents player bullets with velocity components and inherited player horizontal velocity for realistic physics. EnemyBullet represents enemy projectiles with downward movement and runtime transparency masking that removes white backgrounds. Both extend GameObject[1].

Obstacle and PowerUp Classes:

Asteroid extends GameObject for space debris obstacles with configurable speed defaulting to 3.0. The PowerUp abstract class defines a pure virtual apply() method that forces derived classes to implement player interaction. ShieldPowerUp, FireRatePowerUp, and RepairPowerUp each implement specific power-up behaviors[1].

Management Classes:

LevelManager tracks difficulty progression with private members currentlevel, level1score (500), level2score (1500) and methods updateLevel(), getLevel(), getEnemySpeed(), getSpawnInterval() that return difficulty parameters. ScoreManager provides static methods saveScore(), saveBinaryScore(), loadBinaryScores(), displayHighScoresOnImage() for data persistence. ScoreDisplay contains static methods for rendering HUD information[1].

Template Container:

GameObjectArray<T> provides a generic dynamic array for GameObject-derived types with private T** items pointer array, capacity, and count members. Methods include add() for adding items with automatic resizing, updateAll() and drawAll() for iterating through active objects, cleanUp() for removing inactive objects and deleting them, clear() for total cleanup, and get() for indexed access with bounds checking[1].

Game Engine:

GameEngine serves as the central controller using composition to manage all subsystems including sf::RenderWindow window, Player* player pointer (aggregation), multiple GameObjectArray containers (composition), SFML textures/sprites for all screens, sf::Font and sf::Text for rendering, sf::Clock objects for timing, sf::Music for audio, and game state variables[1].

Design Patterns:

Factory pattern is used in GameEngine to create different enemy types depending on the current level. Template Method pattern is used in GameObject where it provides a basic structure that other classes can build upon. Strategy pattern is used in PowerUp classes where each power-up type has its own way of affecting the player. Singleton pattern is used in ScoreManager which uses all static methods for score management. RAII principles ensure automatic cleanup of resources when objects are created and destroyed[1].

UML Diagram

(Can’t be pasted due to resolution issues, it is present in submission folder)

3. Mapping of OOP Concepts to Implemented Features

OOP Concept	Implementation	Code Example
Encapsulation	All the classes we implemented classifies public and private members properly.	Player stores velX, velY, firecooldown as private variables; public methods like getVelocityX() and setFireCooldown() provide controlled access to ensure score and state integrity[1]
Single Inheritance	Player : public Ship; Enemy : public Ship	Both inherit health management from Ship with methods like takeDamage() and getHealth() that are shared across the hierarchy[1]
Multilevel Inheritance	GameObject -> Ship ->Enemy -> FastEnemy (4-level heirarchy)	Each level adds more specific features: GameObject provides rendering and collision, Ship adds health, Enemy adds enemy behavior, and FastEnemy customizes speed and appearance[1]
Polymorphism	virtual update() = 0 in GameObject	This is overridden in Player (for keyboard movement), Enemy (for vertical descent), PatternEnemy (for sine wave movement and shooting), and Projectile (for diagonal motion). Each class provides its own specific behavior that executes through dynamic binding[1]
Dynamic Binding	enemies.updateAll() goes through base pointers calling update()	At runtime, the actual object type determines what gets executed - Enemy does simple descent while PatternEnemy does complex sine wave movement with shooting. Similarly, powerups.get(idx)->apply(*player) executes the correct derived implementation[1]
Abstraction	class GameObject with pure virtual update() = 0	This defines an interface without implementation, making the class non-instantiable; users can only create derived types. The PowerUp class similarly has pure virtual apply() that forces implementations without dictating behavior[1]
Friend Functions	friend void debugPlayerInfo(const Player& p)	This grants a global function access to private members (health, collectedresources, shielded, autoFire) for debugging purposes. friend class ScoreDisplay grants access to private members for HUD rendering[1]
Static Members	static int Enemy::enemycount in Enemy class	This tracks the total active enemies globally across all instances; it gets incremented in the constructor and decremented in the destructor. It can be accessed without an instance: Enemy::enemycount is displayed in the HUD[1]

Static Methods	ScoreManager::saveScore(), saveBinaryScore() all static	These operate on file data without requiring ScoreManager instances; they're called as ScoreManager::saveScore(playerName, score, elapsed) showing a utility class pattern[1]
Const Correctness	getValue() const in Resource; getLevel() const in LevelManager	Const methods guarantee that the object state won't be modified, allowing the compiler to enforce immutability for getters and query methods[1]
Operator Overloading	Resource operator+(const Resource& other) const	This allows natural syntax like collectedresources = collectedresources + r for combining resources. friend ostream& operator<< allows printing with cout << resourceObj[1]
Composition (Has-A Relation)	GameEngine contains GameObjectArray<Projectile> bullets, LevelManager	GameEngine controls the lifetime of these containers; composed objects cannot exist independently, showing a strong ownership relationship[1]
Aggregation (Uses-A Relation)	Player holds GameObjectArray<Projectile>* bullets pointer	Player receives a pointer but doesn't own it; GameEngine owns the array. Player uses the bullets array to spawn projectiles but doesn't control its lifetime, showing a weak relationship[1]
Copy Constructor	Player performs deep copy of healingParticles array	for (int idx = 0; idx < MAX_PARTICLES; idx++) { healingParticles[idx] = other.healingParticles[idx]; } ensures that the copied Player has an independent particle system that doesn't share memory with the original[1]
Deep Copy	Player copy constructor deep copies healingParticles	Element-wise array copying prevents double-delete errors and ensures independent state between copied objects[1]
Virtual Destructor	virtual ~GameObject() {}; virtual ~Enemy() { enemycount--; }	This ensures proper cleanup when deleting derived objects through base pointers; the derived destructor runs first, then the base, allowing proper resource cleanup and static counter updates[1]
Dynamic Memory	enemies.add(new Enemy(...)); Template array uses T** items = new T*[capacity]	Extensive use of the new operator for game objects with corresponding delete operations in cleanUp() and destructors to prevent memory leaks[1]
Pointers	Player* player in GameEngine; GameObjectArray<Projectile>* bullets in Player	These enable polymorphism, dynamic allocation, and indirect access patterns that are essential for the game architecture [1]

Templates	template <typename T> class GameObjectArray	A generic container that works with any GameObject-derived type at instantiation: GameObjectArray<Enemy>, GameObjectArray<Projectile>, eliminating code duplication while maintaining type safety[1]
Exception Handling	throw GameException("Could not save scores"); catch (GameException& err)	Custom exception class thrown on file operation failures, caught in main() with user-friendly error messages to prevent crashes[1]

4. Screenshots of Gameplay and GUI

Game States and Menus



Figure 1: This shows a Splash Screen which is the opening screen that fades out after 4 seconds. Press any key to skip to the main menu[1].

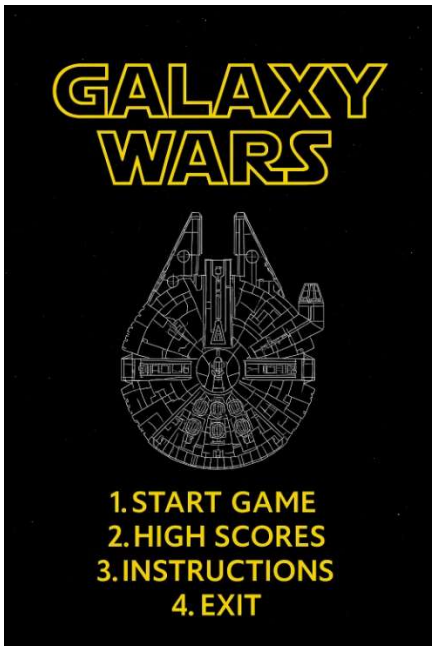


Figure 2: Main Menu: The main menu with four options: Start Game, View Leaderboard, Read Instructions, and Exit Game[1].

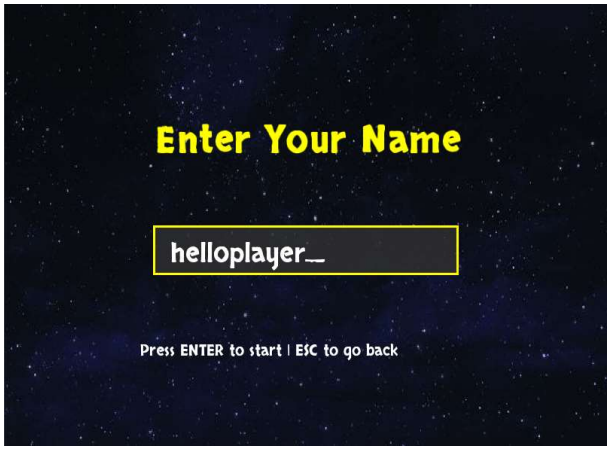


Figure 3: Name Input Screen: It's a screen where you type your name (up to 15 characters). Press ENTER key to start or ESC key to go back[1].

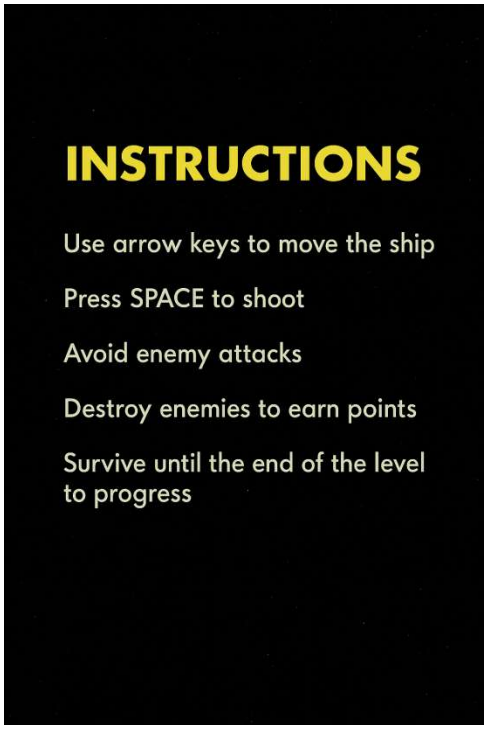


Figure 4: Instructions Page: Shows the game controls, objectives, and what each power-up does. Press ESC key to return[1].



Figure 5: Leaderboard Screen: This shows the top 10 high scores from the scores.txt file. Press ESC to return[1].

Level 1 Gameplay - Score 0-499



Figure 6: Level 1 Start; The player ship starts at the bottom center. The HUD shows Score, HP, Level, and Enemies[1].



Figure 7: Level 1 Combat; Basic enemies come down and you shoot them with bullets. Asteroids also fall. You get 10 points per enemy killed[1].

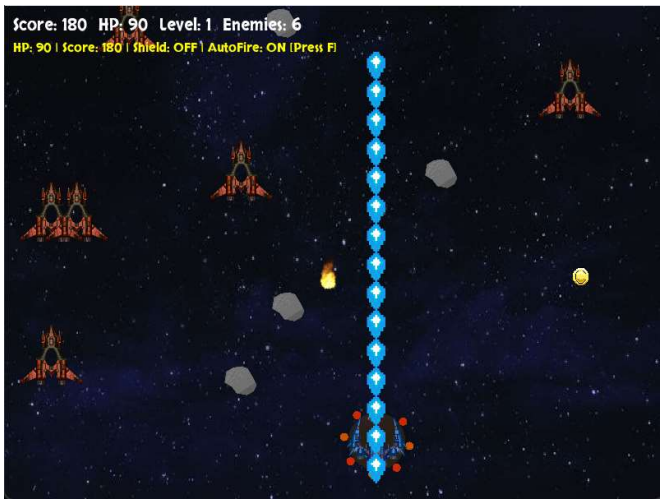


Figure 8: Power-ups; Three power-up types fall down: Shield (blue), Fire Rate (orange), and Repair (green)[1].



Figure 9: Shield Active; A cyan bubble surrounds the player for 5 seconds when shield is active[1].



Figure 10: Level Transition; A screen appears for 5 seconds when you move to Level 2[1].

Level 2 Gameplay (Which has a Score 500-1499)

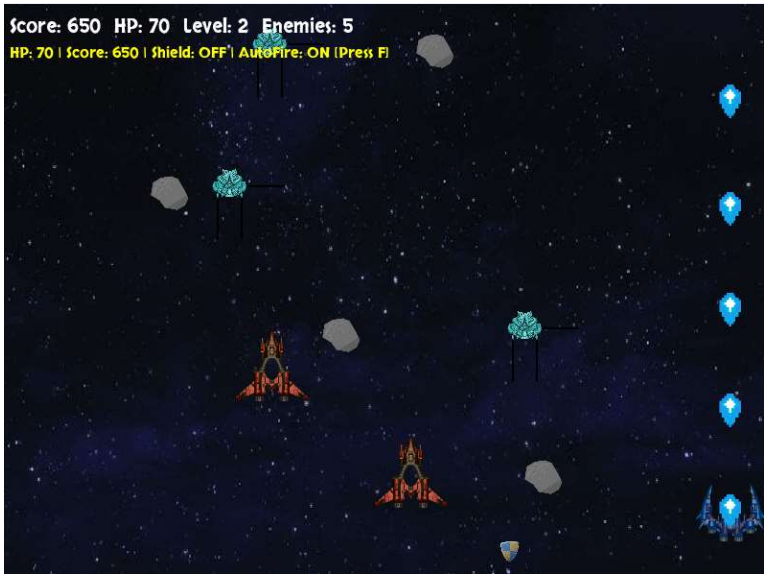


Figure 11: Level 2 Mixed Combat; This includes both basic and fast enemies (red-colored) spawn together. Things get harder here[1].



Figure 12: Level 2 to 3 Transition; Transition screen at 1500 points showing you're entering the final level[1].

Level 3 Gameplay (Score 1500+ and a 120-Second Survival)

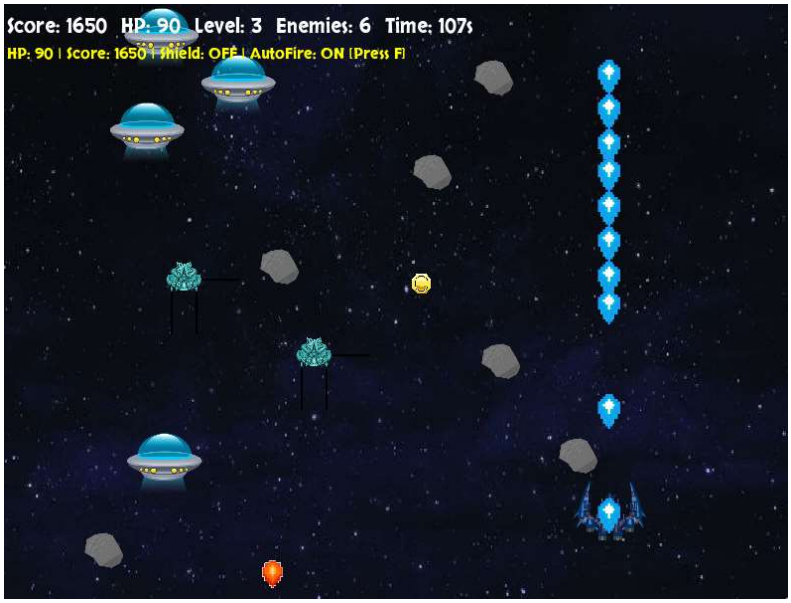


Figure 13: Level 3 Start; on this screen a survival timer appears showing 120 seconds. All three enemy types spawn every 0.3 seconds[1].

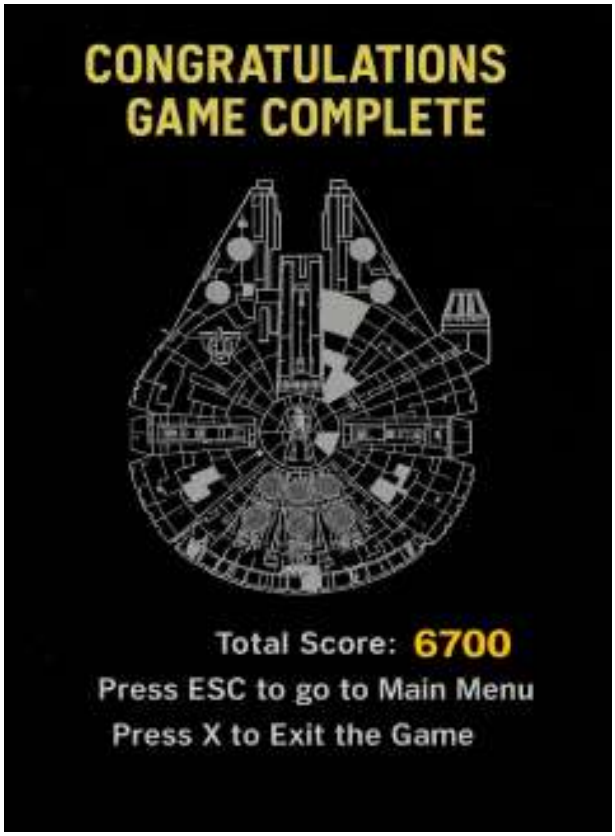


Figure 14: Victory Screen; Shows your final score and time survived when you beat Level 3[1].

Menus:

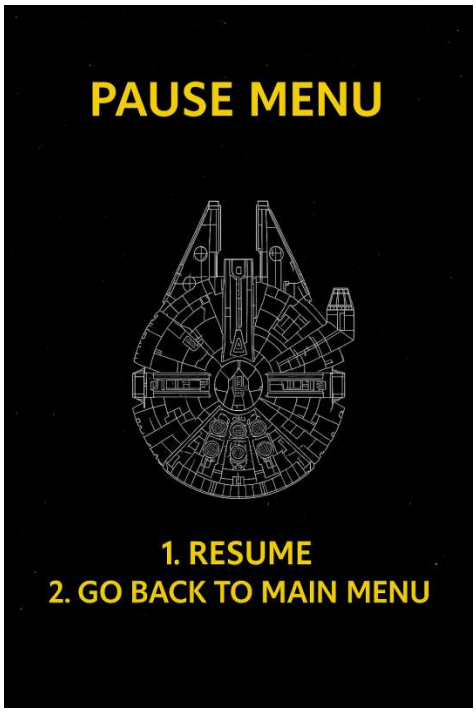


Figure 15: Game Pause Screen



Figure 16: Game Over Screen

5. Challenges and Solutions

Problem 1: Collision Detection Performance

Issue: When we checked collisions between all objects (bullets, enemies, asteroids, powerups, player), we used nested loops that slowed down the game badly. When we had 50+ objects, the frames per second dropped a lot[1].

Solution: We used axis-aligned bounding box collision detection as we mentioned before with a simple BoundingBox struct. We added a cleanUp() method that removes dead objects from arrays right away. We also used an active and inactive flag system to mark destroyed objects without deleting them immediately[1].

Result: We got smooth 60 frames per second even with 100+ objects. Collision checks finish in under 2ms per frame[1].

Problem 2: Game State Management

Issue: Switching between 10 game states (menu, gameplay, pause, level transitions, etc.) caused problems. Music would restart randomly and keyboard inputs would affect the wrong state[1].

Solution: We created a GameState enum and a central updateMusic() method that tracks state changes. Music only restarts when the state actually changes. We added a justEnteredNameInput bool var to prevent accidental character entry. Each state has its own event handling to avoid input mix-ups and lets the game run smooth [1].

Result: Clean state transitions with proper music changes and no input bleeding between states [1].

Problem 3: Enemy Bullets White Background

Issue: The enemy bullet.png had a white background showing which looked bad [1].

Solution: We added runtime color masking in the EnemyBullet constructor using createMaskFromColor() to remove white pixels and make them transparent[1].

Result: Enemy bullets now blend into the space background and look professional [1].

Problem 4: Template Array Memory Leaks

Issue: Our GameObjectArray<T> template had memory leaks when resizing and destroying objects. Memory usage kept growing during long gameplay sessions (from 500KB to several MB) [1].

Solution: We added proper memory management: a destructor that calls clear() and deletes the array, a cleanUp() method that deletes inactive objects right away, and a virtual destructor to ensure derived classes clean up properly[1].

Result: No more memory leaks. The game can run indefinitely without memory growth [1].

Problem 5: File Operation Crashes

Issue: The game would crash when score files were missing, corrupted, or write-protected. There were no error messages or data backup [1].

Solution: We created a custom GameException class and wrapped all file operations in try-catch blocks. When failures happen, we throw meaningful error messages instead of crashing [1].

Result: Robust file handling with graceful error recovery. The game continues even if score saving fails [1].

Problem 6: Visual Effects Performance

Issue: Computing SFML transformations in real-time each frame (shield pulse, fire rotation, particle updates) dropped FPS to 45 when multiple effects were active [1].

Solution: We optimized effects by using pre-calculated sine values for shield pulsing, fixed-size particle arrays instead of dynamic allocation, timer-based updates to reduce calculations, and alpha blending for particles [1].

Result: Visual effects now cost less than 0.5ms per frame with solid 60 frames per second even with all effects enabled [1].

6. Exception Handling and File Input/Output

Exception Handling Strategy

Custom Exception Class:

We made our own GameException class (GameException.h) to handle game-specific errors:

```
class GameException {
   string errormsg;
public:
   GameException(const string& msg) : errormsg(msg) {}
   string getMessage() const { return errormsg; }
};
```

This helps us differentiate game errors from regular C++ errors with clear messages[1].

Exception Throwing:

We throw exceptions when file operations fail:

- ScoreManager::saveScore() throws an exception when it can't open scores.txt
- ScoreManager::saveBinaryScore() throws an exception when it can't open scores.dat
- File read failures also throw exceptions to prevent silent data loss[1]

Exception Catching:

We wrapped the entire game in the main function:

```
try {
    GameEngine game;
    game.run();
} catch (GameException& err) {
    cout << "Game Error: " << err.getMessage() << endl;
} catch (...) {
    cout << "Critical error occurred" << endl;
}
```

This catches our custom game errors with specific messages and also catches any unexpected errors as a backup [1].

File I/O Implementation

Text Format (scores.txt):

We store scores in a simple text format:

- Each line has player name, score, and completion time separated by spaces
- It's easy to read and debug since its in text format
- We use ifstream to read the name, score, and time
- We use ofstream to add new entries [1]

Binary Format (scores.dat):

We also store scores in binary format:

- Name length (4 bytes int) followed by the actual name characters
- Score (4 bytes int) and time (4 bytes float) stored as raw bytes
- This takes up less space then text format

- It needs careful pointer management though [1]

Sorting and Display:

We used bubble sort in ScoreManager::sortScores() to arrange scores from highest to lowest. The top 10 scores show up on the leaderboard screen with nice formatting and colors. Scores save across game sessions so players can see there rankings over time [1].

Error Recovery:

- All file operations check is_open() before doing anything
- We use ios::app mode to safely add to existing files
- Missing files are handled properly with exceptions
- We validate data to prevent corrupted entries from crashing the game [1]

References

[1] Galaxy Wars Source Code and Implementation